

35.2. La fonction `sprite16()` — un personnage de 16×16

`#include <sprite16> : sprite16(numero, x, y, PERSO) place quatre lutins en carré, d'un appel.`

1. Ce qu'il faut comprendre

Ce qu'elle fait : `sprite16(numero, x, y, perso)` place un personnage de 16×16 : les lutins `numero` à `numero + 3`, en carré. Le dessin est un `Perso`, ou seize rangées écrites sur place.

La ligne à écrire : `#include <sprite16>`.

Ses arguments : comme `sprite()` ; le numéro du premier des quatre lutins.

Ce qu'elle coûte : environ **270 octets** avec son dessin.

Essaie : un deuxième personnage avec le numéro 4 : les lutins 4 à 7.

On retrouve `sprite16` dans 2 leçons

[36](#), [47](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **356 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      50 o ← la console, à l'allumage - la mise en route de la console
main()         123 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
RangerLutins   11 o ← démarrage - range les 40 lutins hors de l'écran
InstallerTransfert 15 o ← démarrage - installe la copie des lutins en mémoire rapide
CopierTuiles   31 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
Tuiles         64 o ← CopierTuiles - 0 lettre sur 43, et 4 tuiles dessinées
DonneesTransfert 10 o ← InstallerTransfert - la petite routine qui copie les lutins
```

Lettres gravées : aucune

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `EffacerCarte`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `Notes`

Fonctions du langage appelées :

```
sprite()   x 4   100 o sur place, dans main()
image()    x 1    6 o sur place, dans main() + AttendreImage (28 o)
sprite16() x 1    0 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <sprite16> : un personnage de 16 × 16 ----  
// Un bloc 16 × 16, écrit sur place, qui descend d'un pixel par image.  
  
#include <sprite16>    // sprite16() : quatre lutins en carré  
#include <Perso>       // le dessin de 16 × 16
```

2. UN DESSIN — BLOC

```
Perso BLOC = {  
    "#####", "#.....#", "#.#####.", "#.#####.",  
    "#.#####.", "#.#####.", "#.#####.", "#.#####.",  
    "#.#####.", "#.#####.", "#.#####.", "#.#####.",  
    "#.#####.", "#.#####.", "#.#####.", "#.#####.",  
};
```



à l'écran

3. UNE VARIABLE — Y

```
uint8_t y = 0;
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

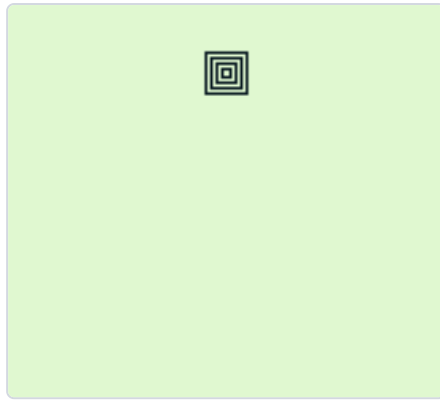
```
int main() {  
    while (true) {  
        image();  
        if (y < 120) y = y + 1;    // il descend, et s'arrête en 120  
        sprite16(0, 72, y, BLOC); // les lutins 0, 1, 2, 3  
    }  
}
```

3. Ce qui se passe, étape par étape

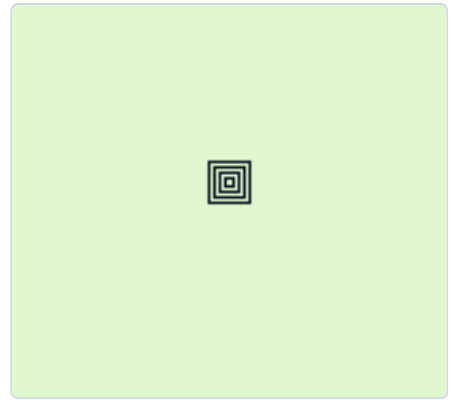
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



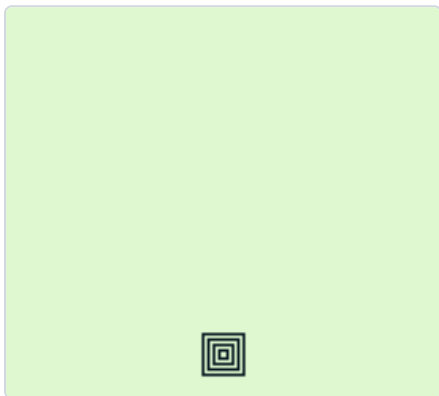
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un bloc carré qui descend au milieu de l'écran, et s'arrête en bas.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le bloc est arrivé en $y = 120$ (120)

✓ les quatre lutins font un carré de 16

Ensuite : **35.3. La fonction `cacher16()` — ôter un personnage de 16×16** — chapitre 6 : Dessiner. · gameboy3 — écrire une cartouche Game Boy en C++